CHIT CREDIT

Parallel 24-Hour Workplan — Hour-by-Hour Tasks & Exact AI Prompts for All 3 Members

Team: Member A (Backend/Data) · Member B (Frontend/Dashboard) · Member C (Product/Integration/Pitch)
Event: CodeChef Innovation Unbound × Temenos — 24-Hour Hackathon

How to use this document: each hour-block below runs in parallel for all three members. Each member's task comes with a ready-to-paste prompt for an AI coding assistant (e.g. Claude Code, Cursor, or this chat). Purple integration boxes mark the exact points where two or more members' work must be merged, and exactly how to do it without conflicts.

Role Summary

Role	Owns	Primary Tools
Member A — Backend/Data	Data model, mock data generator, Chit Ledger API, Chit Credit Score engine, Credit Offer logic	Python/Node backend, SQLite/Firebase, Claude Code
Member B — Frontend/Dashboard	Member dashboard, operator dashboard, charts, UI polish, Tamil string layer	React/HTML-CSS-JS, Chart.js/Recharts, Claude Code / Claude Design
Member C — Product/Integration/Pitch	Income smoothing logic, elastic repayment logic, WhatsApp/notification mock, deck, demo script, rehearsal	Python/Node glue code, PowerPoint/Claude, Claude Code

Golden rule for parallel work: agree on the data contract (field names, JSON shape) in Hour 0 and never change a field name unilaterally afterward — every integration conflict below is prevented by this one discipline.

HOUR 0 – 1 | Kickoff, Data Contract, Repo Setup

Member A

Define the shared data model (Member, DailyEarning, ChitContribution, ChitCreditScore, CreditOffer, LoanRepayment) and set up the repo with folders for backend, frontend, and scripts.

Prompt to give your AI coding assistant:

```
Set up a new project repo for a hackathon fintech app called Chit Credit. Create a backend/ folder (Node+Express or Python+FastAPI, pick the faster option), a frontend/ folder (React), and a scripts/ folder. Define these entities as a shared schema file (JSON schema or TypeScript types): Member, DailyEarning, ChitContribution, ChitCreditScore, CreditOffer, LoanRepayment. Fields: [paste fields from Section 6.2 of the blueprint]. Output the schema as a single schema.json file both frontend and backend can reference.
```

Member B

Wireframe the member dashboard and operator dashboard on paper/Figma-style text first, agree on screen list with Member C before building anything.

Prompt to give your AI coding assistant:

```
I'm building a 2-screen fintech dashboard for a hackathon: (1) a Member Dashboard showing chit credit score, tier badge, contribution history table, and an income-smoothing chart, and (2) an Operator Dashboard showing chit group health, member list, and default/fraud flags. Give me a simple wireframe description (sections top to bottom) for both screens, optimized for a fast 24-hour build in React with Tailwind CSS.
```

Member C

Confirm and write down the real-world facts to anchor the pitch (IppoPay, CredRight, chit fund statistics) and lock the one-line pitch and problem framing the team will repeat all day.

Prompt to give your AI coding assistant:

```
Search for the latest verified statistics on: (1) number of registered chit fund subscribers in India, (2) IppoPay as a QR/UPI payments company serving informal merchants and chit fund operators in Tamil Nadu, (3) CredRight's work digitizing chit funds (referenced by Accion.org). Summarize each in 2 sentences with source names, so I can cite them confidently in a hackathon pitch deck.
```

INTEGRATION CHECKPOINT

By the end of Hour 1, all three members must have READ and AGREED on Member A's schema.json. Member B builds UI components against these exact field names. Member C's smoothing/repayment logic must also reference these field names. If a field needs to change after this point, it must be announced to all three in the team chat immediately, not silently changed.

HOUR 1 – 3 | Foundations

Member A

Build the mock data generator: 15-20 members, jagged daily earnings for 30 days, and a monthly chit due date per member.

Prompt to give your AI coding assistant:

```
Write a Python script that generates realistic mock data for 18 members of a chit fund app: for each member, generate 30 days of jagged daily earnings (gig/vendor income, ranging roughly 200-1500 INR with random dips simulating slow days), and one monthly chit contribution due date with a fixed amount_due. Make 4 members have a poor payment history (missed/late), 4 have excellent history, and the rest average. Output as JSON files matching this schema: [paste schema.json]. Save to /data/mock_members.json and /data/mock_earnings.json.
```

Member B

Scaffold the React app: routing, layout shell, and the chart library setup so wiring in real data later is fast.

Prompt to give your AI coding assistant:

```
Scaffold a React app with Tailwind CSS for a hackathon fintech dashboard. Set up React Router with two routes: /member/:id (Member Dashboard) and /operator (Operator Dashboard). Install and configure Recharts. Build a shared Layout component with a simple navy-and-gold color theme (navy #0B2447, gold #B98B2A). Leave placeholder sections matching this wireframe: [paste wireframe from Hour 0-1].
```

Member C

Draft the Chit Credit Score weighting logic in plain language/pseudocode together with Member A's data shape, so Member A can implement it in Hour 3-6.

Prompt to give your AI coding assistant:

```
Help me design a simple, explainable rule-based credit scoring formula for a 'Chit Credit Score' using these inputs: on_time_payment_percentage, contribution_streak_length, tenure_in_completed_cycles, missed_payment_count. Give me a weighted formula, suggested weights, and 3 score bands (Bronze/Silver/Gold) with example threshold values, written as pseudocode I can hand to a backend developer.
```

INTEGRATION CHECKPOINT

No merge yet, but Member C must hand the finalized scoring pseudocode to Member A by end of Hour 3 so Member A can start building the scoring engine in the next block without waiting.

HOUR 3 – 6 | Core Engines

Member A

Build the Chit Ledger API and the Chit Credit Score engine using Member C's formula, tested against the mock data.

Prompt to give your AI coding assistant:

```
Using this schema [paste schema.json] and this mock data [paste mock_members.json, mock_earnings.json], build a backend API (Express or FastAPI) with endpoints: POST /contribution (log a chit payment), GET /member/:id/score (returns Chit Credit Score using this formula: [paste Member C's pseudocode]), GET /member/:id/history (contribution history). Return score banded into Bronze/Silver/Gold. Include basic input validation and sample curl requests to test each endpoint.
```

Member B

Build the Member Dashboard UI: score display, tier badge, and contribution history table, using dummy hardcoded data for now.

Prompt to give your AI coding assistant:

```
Build a React component called MemberDashboard that displays: a large Chit Credit Score number, a colored tier badge (Bronze/Silver/Gold), and a table of past chit contributions (date, amount, on-time yes/no). Use this dummy data for now: [paste 1 sample member JSON]. Style with Tailwind CSS, navy and gold theme, mobile-responsive. Leave a clearly marked TODO comment where real API data will be wired in later.
```

Member C

Start building the Income Smoothing engine: variable daily reserve percentage logic that tops up a wallet toward the fixed chit contribution.

Prompt to give your AI coding assistant:

```
Write a function (Python or JS, match Member A's backend language) called calculateSmoothingReserve(dailyEarnings: array, trailingWindow: 7, basePct: 0.15) that: for each day, reserves a higher percentage into a wallet if that day's earning is above the member's trailing 7-day average, and a lower percentage if below. Return a running wallet balance array and flag the date it reaches the fixed chit contribution amount, simulating auto-payment on the due date. Include a simple test using this mock earnings data: [paste sample 30-day earnings array].
```

INTEGRATION CHECKPOINT

End of Hour 6: Member A's score engine and Member C's smoothing engine must both run successfully against the SAME mock member (pick member_id = 1 as the shared test case) so results can be sanity-checked together before Member B starts wiring real API calls in the next block.

HOUR 6 – 9 | Feature Build-Out

Member A

Build the Credit Offer unlock logic: mapping score bands to a mock NBFC loan offer (amount, interest rate).

Prompt to give your AI coding assistant:

```
Add a new endpoint GET /member/:id/credit-offer to the backend that takes the member's Chit Credit Score band (Bronze/Silver/Gold) and returns a mock NBFC credit offer object: { eligible_amount, interest_rate, partner_nbfc }. Bronze = 5000 INR at 18%, Silver = 15000 INR at 14%, Gold = 30000 INR at 11%. Return 'not yet eligible' for members below Bronze threshold. Add this to the schema and API docs.
```

Member B

Build the income-smoothing visual: a chart showing raw jagged daily earnings vs. the smoothed contribution line, this is the single most important demo visual.

Prompt to give your AI coding assistant:

```
Build a React component using Recharts called SmoothingChart that renders two overlaid lines on one chart: (1) raw daily earnings (jagged, red/orange line) and (2) the smoothed wallet balance building toward the fixed contribution amount (smooth, green line), using this data shape: [paste Member C's smoothing engine output format]. Add a horizontal dashed line marking the due-date contribution target. Make it visually striking since this is the hackathon demo's key visual moment.
```

Member C

Build the Elastic Repayment simulator logic: daily deduction as a percentage of earnings, shrinking automatically on low-earning days.

Prompt to give your AI coding assistant:

```
Write a function calculateElasticRepayment(dailyEarnings: array, loanBalance: number, deductionPct: 0.10) that, for each day, deducts min(dailyEarning * deductionPct, remainingBalance) from the loan balance and returns a day-by-day repayment schedule array with remaining balance. Also add a small 7-day moving-average trend check that lowers deductionPct by half automatically if a predicted slow period is detected (stretch goal, keep this part simple and clearly commented as optional).
```

INTEGRATION CHECKPOINT

Nothing to merge yet structurally, but Member B needs Member C's exact output JSON shape for the SmoothingChart by hour 7 latest, and Member A's credit-offer response shape needs to be shared with Member C before Hour 9 so the repayment simulator can pull a realistic loan_balance to start from.

HOUR 9 – 12 | Full Backend-Frontend Wiring (First Integration)

Member A

Connect all API endpoints together, run through all 18 mock members end-to-end, fix data edge cases (e.g. zero contributions, brand-new members).

Prompt to give your AI coding assistant:

```
Review my backend API [paste current API file] against this mock dataset of 18 members [paste mock data]. Run through every member and flag any case that would break the score engine, credit-offer logic, or smoothing calculation (e.g. a member with zero contributions, a member with only 1 day of earnings, negative balances). Fix each edge case and add basic error handling so the API never crashes on the demo dataset.
```

Member B

Replace all dummy data in the Member Dashboard and Operator Dashboard with real API calls to Member A's backend; build the Operator Dashboard's chit-group health view.

Prompt to give your AI coding assistant:

```
Wire my React MemberDashboard component [paste component] to call the real backend endpoints: GET /member/:id/score, GET /member/:id/history, GET /member/:id/credit-offer, at base URL [paste Member A's local server URL]. Handle loading and error states. Then build a new OperatorDashboard component showing: a list of all members in a chit group, their score tier, and a red flag icon for any member with 2+ missed payments, using GET /group/:id/members (ask Member A to add this endpoint if missing).
```

Member C

Build the WhatsApp/notification mock layer and start integrating the smoothing engine's output into the actual API flow (auto-pay simulation) rather than a standalone script.

Prompt to give your AI coding assistant:

```
I need a simple simulated notification system for a hackathon demo (no real WhatsApp API access needed if time is short): build a NotificationFeed React component that shows mock WhatsApp-style message bubbles in Tamil and English, e.g. 'Your chit payment of Rs 500 was auto-paid from your savings wallet on time!' Trigger a new message whenever the smoothing engine's wallet reaches the due-date target. Connect this to Member A's contribution API so a real auto-pay event fires the notification.
```

INTEGRATION CHECKPOINT

THIS IS THE FIRST MAJOR INTEGRATION POINT. All three members sit together for the last 30 minutes of this block: Member B's dashboards must be showing REAL data from Member A's API, and Member C's notification feed must fire from a REAL contribution event, not a mock timer. If any API field name mismatch appears, fix it now — do not carry mismatches into the sleep block.

HOUR 12 – 15 | Rest / Buffer Block (Staggered)

Member A

Rest. Optional: leave a short list of known bugs/TODOs in a shared doc for the next block.

Prompt to give your AI coding assistant:

```
(No prompt needed — but before resting, write a 5-line TODO list of any known bugs or edge cases still unresolved in the backend, so the team can triage them immediately after the rest block.)
```

Member B

Rest, staggered with Member A if possible so one person is reachable for quick fixes.

Prompt to give your AI coding assistant:

```
(No prompt needed — before resting, write a 5-line TODO list of any UI issues or missing states discovered while wiring real data.)
```

Member C

Use part of this block (if not resting) to start the deck skeleton, since this task does not require the other two members' code to be finished.

Prompt to give your AI coding assistant:

```
Create a 7-slide outline for a hackathon pitch deck for 'Chit Credit', using this structure: Hook, The Insight, The Solution (3-layer diagram), Live Demo transition, Real-World Anchor, Impact & Sustainability, Close. For each slide, give me a one-line headline and 2-3 supporting bullet points, in a confident but not overhyped tone, based on this project summary: [paste Section 1-5 of the Chit Credit blueprint].
```

INTEGRATION CHECKPOINT

No integration needed in this block by design — this is intentional slack time. If the team is ahead of schedule, skip straight to Hour 15-17 early rather than force a full 3-hour rest.

HOUR 15 – 17 | Full Integration Push

Member A

Triage and fix all backend TODOs from the rest block; ensure the API is stable for the remaining build hours.

Prompt to give your AI coding assistant:

```
Here are the known backend TODOs from before the break: [paste TODO list]. Go through each one, propose a fix, and implement it. After fixing, run through all 18 mock members once more end-to-end to confirm nothing regressed.
```

Member B

Triage and fix all frontend TODOs; make sure both dashboards render cleanly with zero console errors against real data.

Prompt to give your AI coding assistant:

```
Here are the known frontend TODOs from before the break: [paste TODO list]. Fix each one. Then do a full pass on both MemberDashboard and OperatorDashboard checking for: console errors, broken layout on smaller screens, and any leftover hardcoded dummy data that should now be using the real API.
```

Member C

Build the Elastic Repayment slider UI (interactive demo piece) and connect it live to Member C's own repayment engine and Member A's credit-offer data.

Prompt to give your AI coding assistant:

```
Build a React component called RepaymentSimulator with a slider labeled 'Today's Earnings (INR)'. As the user drags the slider, show the elastic repayment deduction amount updating live using this function: [paste calculateElasticRepayment]. Pull the starting loan_balance from Member A's GET /member/:id/credit-offer endpoint. Make the deduction number animate/highlight when it changes so it reads well on a projector during the live demo.
```

INTEGRATION CHECKPOINT

By the end of Hour 17, run a full click-through as a team: Member B drives the browser through both dashboards while Member A watches the backend logs and Member C watches for any smoothing/repayment calculation that looks wrong on screen. Fix anything broken together before splitting up again.

HOUR 17 – 19 | Polish + Content

Member A

Seed final, realistic demo data (15-20 members with a good spread of tiers/stories) so the live demo tells a clear narrative.

Prompt to give your AI coding assistant:

```
Refine my mock data generator so it produces a specific, demo-friendly set of members: 'Meena' (Gold tier, 3-year streak, never won a bid), 'Raghu' (jagged gig income, mid-tier, benefits visibly from smoothing), and 'Selvam' (operator view test case with 2 members in default). Regenerate mock_members.json and mock_earnings.json with these named personas as the first 3 entries so the demo can reference them by name.
```

Member B

Final UI polish pass: responsive check, consistent spacing/colors, loading states, and adding the Tamil-language string layer to 3-4 key screens.

Prompt to give your AI coding assistant:

```
Add a simple language toggle (English/Tamil) to my MemberDashboard component. Store UI strings (score label, tier names, contribution history header, notification text) in a small i18n object with English and Tamil translations for these 4 screens: [list screens]. Keep it lightweight — no full i18n library needed, just a strings lookup object and a toggle button.
```

Member C

Finalize the deck content with real numbers and screenshots once Member B's UI is stable; write the full demo script with exact timings.

Prompt to give your AI coding assistant:

```
Using this deck outline [paste outline] and these final statistics [paste verified stats], write full slide copy for all 7 slides, plus a timed demo script broken into: 0:00-0:40 hook, 0:40-1:30 Layer 1 live, 1:30-2:30 Layer 2 live, 2:30-3:15 Layer 3 live, 3:15-3:45 real-world anchor, 3:45-4:15 impact and close. Include the exact one-line pitch closing statement.
```

INTEGRATION CHECKPOINT

Member C needs Member B's final screenshots/UI by hour 18 to drop into the deck slides — do a quick screen-share handoff rather than waiting for a formal file export.

HOUR 19 – 21 | Stretch Goals + QA

Member A

If time allows: add the simple predictive slow-season forecast stub. If not, skip and move to QA support for Member B/C.

Prompt to give your AI coding assistant:

```
Add an optional endpoint GET /member/:id/forecast that computes a 7-day moving average trend on daily earnings and flags 'slow period predicted' if the trend is declining for 3+ consecutive days. Keep this simple — no ML model, just moving-average comparison. Clearly comment it as a stretch-goal feature.
```

Member B

If time allows: overlay the forecast flag visually on the SmoothingChart. If not, focus fully on bug fixing and cross-browser check.

Prompt to give your AI coding assistant:

```
If Member A's GET /member/:id/forecast endpoint returns a 'slow period predicted' flag, add a small warning badge overlay on the SmoothingChart component near the relevant date range. Keep this as a non-blocking visual addition — the chart must still render correctly if this data is absent.
```

Member C

Run a full dry-run pitch rehearsal using the near-final product; time it strictly and note any moment that runs long or falls flat.

Prompt to give your AI coding assistant:

```
(No AI prompt needed — this is a live team rehearsal. Time the full demo script against a stopwatch, run through all 3 live-demo segments on the actual built product, and write down any slide or transition that needs tightening.)
```

INTEGRATION CHECKPOINT

After the rehearsal, the team reconvenes for 10 minutes to agree on any last cuts — if the forecast stretch goal is shaky, agree explicitly to either fix it fast or drop it from the demo entirely. Do not leave this ambiguous going into the final hours.

HOUR 21 – 22 | Convergence

Member A

Final backend freeze prep: fix only what the rehearsal flagged as broken, nothing new.

Prompt to give your AI coding assistant:

```
Here is the list of issues found during rehearsal: [paste list]. Fix only these, and only in the backend. Do not add any new features at this stage. After fixing, re-run the full 18-member data check one final time.
```

Member B

Final frontend freeze prep: fix only what the rehearsal flagged, do a final visual polish pass.

Prompt to give your AI coding assistant:

```
Here is the list of UI issues found during rehearsal: [paste list]. Fix only these. Do one final pass checking font sizes and color contrast will read clearly on a projector screen from a distance.
```

Member C

Finalize deck design, add speaker notes, and prepare a backup slide (screenshots of the app) in case live demo internet/device issues occur.

Prompt to give your AI coding assistant:

```
Take my final deck content [paste deck copy] and format it into clean, presentation-ready slide text with speaker notes under each slide. Also draft one backup 'screenshot-only' slide summarizing all 3 live-demo moments as static images, in case we need to skip the live demo due to technical issues.
```

INTEGRATION CHECKPOINT

Team reconvenes for a final full run-through of the ENTIRE product plus the ENTIRE pitch back-to-back, exactly as it will be presented to judges, no stopping to fix anything mid-run. Note issues, fix immediately after in the next block.

HOUR 22 – 23 | Freeze & Final QA

Member A

Code freeze. Only critical, demo-breaking bugs get fixed. Deploy/host the backend if applicable.

Prompt to give your AI coding assistant:

```
Here is my current backend codebase [paste/summarize]. Do a final review for any error that would visibly break during a live demo (crashes, undefined values shown on screen, slow response times). Fix only demo-breaking issues. If deploying, walk me through the fastest way to host this (e.g. Render, Railway, or simply running locally with a stable local network demo).
```

Member B

Code freeze. Only critical, demo-breaking bugs get fixed. Confirm the app works on the exact device/browser that will be used for the live demo.

Prompt to give your AI coding assistant:

```
Here is my current frontend codebase [paste/summarize]. Test it specifically on [demo laptop/browser]. Fix only issues that would visibly break during the live demo. Do not refactor or restyle anything non-critical at this stage.
```

Member C

Finalize and submit the deck; do one last full timed rehearsal solo, then with the team.

Prompt to give your AI coding assistant:

```
(No AI prompt needed — finalize deck file, upload/submit per hackathon portal instructions, and do one final solo timed read-through of the demo script before the last team rehearsal.)
```

INTEGRATION CHECKPOINT

This is the last point where all three members' work must be simultaneously stable together. After this hour, treat the product as untouchable except for true emergencies.

HOUR 23 – 24 | Buffer & Submission

Member A

Standby for last-minute fixes only. Confirm backend is running and stable for the judging window.

Prompt to give your AI coding assistant:

```
(No AI prompt needed — standby role. If an emergency bug appears, fix minimally and re-test immediately against the exact demo flow, nothing else.)
```

Member B

Standby for last-minute fixes only. Confirm frontend is running and stable for the judging window.

Prompt to give your AI coding assistant:

```
(No AI prompt needed — standby role, same as Member A.)
```

Member C

Submit all final files per the hackathon portal requirements, confirm team members' names/roles are correctly listed, do a final calm read of the pitch.

Prompt to give your AI coding assistant:

```
(No AI prompt needed — administrative submission task. Double-check file formats, team registration details, and submission deadline against the official CodeChef Innovation Unbound portal instructions.)
```

INTEGRATION CHECKPOINT

Final integration checkpoint: one member does a last cold-start test of the ENTIRE product from a freshly opened browser/terminal, exactly as a judge would experience it, with zero prior context. If it works cleanly cold, you are done.

Integration Principles Recap

- **Lock the data contract in Hour 0.** Every integration conflict in this plan traces back to a mismatched field name or data shape — agree once, change loudly if ever needed.
- **Integrate early and often, not just at the end.** This plan has 5 explicit integration checkpoints (Hours 1, 6, 9, 17, 21, 22) rather than one big merge at Hour 23 — small frequent merges surface bugs while there is still time to fix them.
- **Member A's core (Layer 1: Ledger + Score) is the non-negotiable backbone.** If time runs short anywhere in the plan, cut polish from Member B's stretch visuals or Member C's forecast stretch goal first — never cut into Member A's core score engine.
- **Use a shared branch or shared local server, not isolated silos.** Member B should be pointing at Member A's real (even if incomplete) API from Hour 9 onward, not building against permanent dummy data — this surfaces integration bugs 12+ hours before the deadline instead of 1 hour before.
- **Every rest/buffer block is intentional slack, not wasted time.** If the team is ahead of schedule, pull forward from the next block rather than idling — the plan is a guide, not a rigid clock.

End of document. Each hour-block's prompts can be pasted directly into an AI coding assistant, with the bracketed placeholders (e.g. [paste schema.json]) filled in with your team's actual current files.